

NAN 2026 · NHN GAME × AI HACKATHON

AI를 도구가 아니라 개발 방법론으로 썼습니다

AI-DLC로 12일 만에 만든 게임 — SIN DIVE

제출 항목	04. AI 활용 기술 문서
방법론	AI-Driven Development Life Cycle (AWS)
결과	라이브 배포 · 테스트 1,657개 · 커밋 131개
작성일	2026-08-09

요약 — 세 문장

1. 저희는 게임을 만들면서 AI를 **도구로 쓰지 않고, 개발 방법론으로 썼습니다**. AWS가 제안한 **AI-DLC(AI-Driven Development Life Cycle)**를 게임 개발에 그대로 적용했습니다.
2. 그 결과가 **12일 만에 나온 완성된 게임**입니다 — 라이브 배포, 테스트 1,657개 통과, 커밋 131개, 코드 11만 라인.
3. 핵심은 "AI가 코드를 써 줬다"가 아닙니다. **AI가 계획을 세우고 사람이 결정을 내리는 순환**을 12일 동안 12번 돌렸고, 그 기록이 전부 레포에 남아 있습니다.

12일

기획 → 라이브 배포

1,657

통과한 테스트

131

커밋

16

설계 문서

1. 왜 AI-DLC인가

1-1. AI 코딩의 진짜 문제

AI로 게임을 만들어 본 팀이라면 다 겪는 일이 있습니다. **처음 3일은 놀랍고, 그다음 3일은 무너집니다**. 코드가 빠르게 늘어나는데 어디가 왜 그렇게 됐는지 아무도 모르게 되고, 고치면 다른 데가 깨집니다. AI가 느려서가 아니라 **결정이 기록되지 않아서** 그렇습니다.

AI-DLC는 이 문제를 정면으로 다룹니다. AWS의 정의를 그대로 옮기면 이렇습니다.

"AI가 계획을 만들고, 확인을 요청하고, 계획을 실행한다. 사람은 중요한 결정을 내린다."

— AWS, AI-Driven Development Life Cycle

AI를 자율 에이전트로 풀어놓는 것도, 사람이 한 줄씩 검토하는 것도 아닙니다. **AI가 속도를 내고 사람이 방향을 줍니다**. 저희에게 필요했던 것이 정확히 이것이었었습니다.

1-2. 세 가지 개념만 알면 됩니다

AI-DLC 개념	뜻	기존 방식에서
볼트 (Bolt)	몇 시간 ~ 하루 단위의 짧은 실행 주기	스프린트 (2주)
작업 단위	하나로 묶여 완결되는 기능 덩어리	에픽
영속 맥락	결정 · 계획 · 산출물을 레포에 남깁니다	사람 머릿속 / 회의록

스프린트가 볼트가 되는 것이 체감 지점입니다

2주 단위로 계획하면 AI의 속도가 계획에 갑니다. 하루 단위로 끊으면 **아침에 정한 것을 저녁에 화면으로 봅니다**.

1-3. 세 단계

INCEPTION 무엇을 · 왜	CONSTRUCTION 어떻게	OPERATIONS 배포 · 운영
- 요구사항 정리 - 설계 문서 작성 - 위험 · 복잡도 평가	- 컴포넌트 설계 - 코드 생성 - 테스트 전략	- 인프라 코드 - 자동 배포 - 모니터링

▲ 각 단계 사이에 **사람의 결정 게이트**가 있습니다

2. 저희가 실제로 어떻게 돌렸는가

2-1. Inception — 코드를 쓰기 전에 문서를 먼저

규칙

기능 하나마다 설계 문서 하나. 문서 승인 없이는 코드를 쓰지 않습니다.

AI가 초안을 쓰고, 선택이 갈리는 지점마다 **사람에게 물었습니다**. 실제로 레포에 남은 문서는 다음과 같습니다.

문서	무엇을 정했나
<code>docs/specs/2026-07-27-ux/</code> (11개)	전 화면 UI/UX 구조, 아트 디렉션, 씬별 명세
<code>2026-07-24-pvp-design.md</code>	PvP 기본 구조
<code>2026-07-26-pvp-mvp-design.md</code>	PvP 최소 구현 범위
<code>2026-08-07-female-recolor-design.md</code>	캐릭터 리컬러 파이프라인
<code>2026-08-08-four-roster-solo-bgm-design.md</code>	로스터 4종 · 싱글 1인 · BGM 분리

이것이 AI-DLC가 말하는 **Mob Elaboration**입니다 — AI가 질문과 제안을 내고, 팀이 그것을 검증합니다. 저희는 소규모 팀이므로 실시간 협업 세션 대신 **문서 + 선택지 질문**으로 같은 일을 했습니다.

2-2. 결정 게이트 — AI가 고르지 않은 것들

이 문서에서 가장 중요한 표입니다. AI가 코드를 얼마나 썼는지가 아니라, **사람이 무엇을 결정했는지가** 이 방법론의 증거입니다.

갈림길	AI가 낸 선택지	사람이 고른 것	왜 사람이 골라야 했나
AUTO를 끄면 어디까지 멈추나	① 동작만 ② 동작+데미지 ③ 표시만	② 동작+데미지	"게임이 뭘 약속하는가"의 문제라 코드로 유도할 수 없습니다
싱글에서 카드를 어떻게 주나	① 층마다 ② 확률 ③ 10층마다 1장	③ 10층마다 1장	수집 속도는 재미의 문제입니다
카드함을 어디에 두나	① 타이틀 우상단 ② 하단 탭 ③ 메뉴 안	① 우상단 작은 버튼	메인 버튼의 위계를 건드릴 수 없습니다
무기와 외형이 어긋날 때	① 외형 우선 ② 기획서 무기 우선	① 외형 우선	화면에 보이는 쪽이 이깁니다
BGM이 음량 조건에 걸릴 때	① 상수 조정 ② 곡 교체	② 곡 교체	조정으로 풀 수 있는 문제가 아니었습니다

2-3. Construction — 코드는 AI, 판정은 화면

AI가 코드를 쓰는 것은 이제 놀랍지 않습니다. 문제는 "됐다"를 누가 판정하느냐입니다. 그래서 규칙을 하나 세웠습니다 — **결과는 찍어서 이미지로 봅니다.**



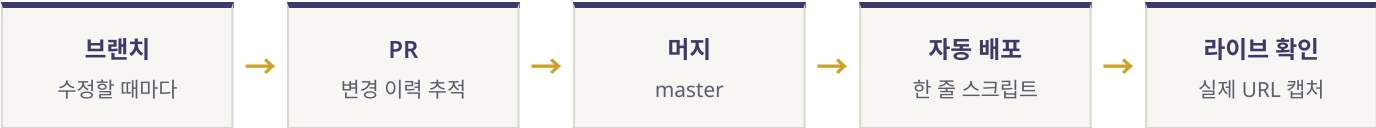
`tools/shot.mjs`가 실제 브라우저를 띄워 **실제 크기 그대로** 캡처합니다. 축소 캡처는 도트 그래픽에서 획을 뭉개기 때문에 판정에 쓸 수 없습니다.

왜 이 규칙이 필요했는가 — 실패 기록

사건	지표는	화면은
죽은 몬스터 HP 바	테스트 전부 통과	죽은 적의 HP 바가 화면에 남아 아래로 흘러내렸습니다
AUTO 배지	테스트 전부 통과	<code>AUTO OFF</code> 라고 적힌 채 캐릭터가 싸웠습니다
캐릭터 상하 반전	설계 문서 근거 있음	캡처를 보니 거울상이었습니다

세 번 모두 테스트는 초록불이었습니다
그래서 "테스트 통과"를 완료 조건으로 쓰지 않습니다.

2-4. Operations — 고치면 그날 배포합니다



한 사이클이 수습 분입니다. 오늘(8/9) 유저 신고 4건을 받아 **같은 날 고쳐서 배포하고 라이브 화면으로 확인**했습니다. 배포는 `infra/deploy-ecs.sh` 한 줄이고, 컨테이너 빌드 → ECR 푸시 → ECS 롤아웃 → CDN 캐시 무효화가 자동으로 이어집니다.

3. 사용한 AI 도구

3-1. 코드

도구	용도
Claude (Anthropic)	설계 문서, 코드 생성, 테스트 작성, 리팩터링, 검증 하네스
Claude Code (CLI)	레포 안에서 직접 파일을 읽고 고치는 실행 환경

3-2. 이미지 — Amazon Bedrock

모델	용도	산출물
<code>stability.sd3-5-large</code>	캐릭터 삽화	인물 21장 (4종 × 카드 · 선택 · 승리)
<code>stability.sd3-5-large</code>	배경 삽화	지상 · 심연 테마별 3겹
<code>stable-image-remove-background</code>	배경 제거	인물 컷아웃

생성은 `tools/gen_portraits.py` · `tools/gen_bg_art.py` 가 맡습니다. **손으로 그린 것이 아니라 스크립트가 다시 굽습니다** — 프롬프트를 고치면 전부 다시 나옵니다.

3-3. 게임 안의 AI — PvP 상대 전략

`vite-plugin-bedrock.ts` 에서 **Claude Haiku 4.5**를 호출해 PvP 팀원의 스킬 판단을 만듭니다. 개발 환경에서 전략 프리셋을 유도하고, 배포 빌드는 그 결과를 프리셋으로 담아 씁니다 — **브라우저에 자격증명이 나가지 않게** 하려는 설계입니다.

4. 프롬프트 — 실제로 쓴 것

요약이 아니라 **원문 그대로** 실습니다. 심사자께서 검증할 수 있어야 근거가 됩니다.

4-1. 캐릭터 삽화 (SD3.5 Large)

```
pixel art portrait of a female athlete, {색·의상 서술}, holding {무기},
side view, dark fantasy, 32-bit pixel art, black background,
full body, standing, arms away from torso
```

4-2. 프롬프트에서 배운 것 세 가지

이 절이 이 문서에서 가장 실용적인 부분입니다. 프롬프트는 한 번에 맞지 않았습니다.

시도	무엇이 문제였나	무엇으로 풀었나
<code>knight</code> , <code>mage</code> 같은 직업 명사	의상 · 배경까지 끌고 와서 갑옷과 성이 생겼습니다	직업 명사를 빼고 <code>athlete</code> 만 남겼습니다
"주먹을 쥐어라"를 세 번 서술	세 번 다 손이 퍼진 채 나왔습니다	무기를 들리자 6/6 주먹이 됐습니다
팔을 몸에 붙인 포즈	리깅 시 손 관절이 잡을 점이 0개였습니다	<code>arms away from torso</code> 를 명시했습니다

두 번째가 특히 중요합니다

AI에게 결과를 서술해도 안 되면, 그 결과가 나오게 되는 상황을 주는 것이 빠릅니다. 주먹을 쥐라고 말하는 대신 질 것을 썼습니다.

4-3. 실패해서 버린 것 — 3D 메시 리깅

캐릭터를 3D로 만들어 자동 리깅하려 했습니다. 43개 동작 중 16개만 통과해서 폐기했습니다. 원인은 리깅 도구가 아니라 입력 원화의 포즈였습니다 — 팔이 몸에 붙어 있으면 손 관절이 잡을 정점이 없습니다. 대신 2D 스프라이트 + 색 재배치로 전환했습니다.

실패를 적는 이유가 있습니다

"AI로 다 됐다"는 문서는 검증할 것이 없습니다. 무엇이 왜 안 됐는지가 그 팀이 실제로 해 봤다는 증거입니다.

5. 결과 — 12일, 12볼트

5-1. 숫자

항목	값
개발 기간	2026-07-24 ~ 08-09 (12 작업일)
볼트 (실행 주기)	12회 (하루 단위)
커밋	131개
추가된 코드	117,686 라인
소스 파일	147개 (TypeScript)
테스트	1,657개 (91개 파일) — 전부 통과
설계 문서	16개
AI 생성 에셋	인물 21장 + 배경 6겹 + 카드 30장
배포	라이브 (CloudFront + ECS Fargate + AppSync)

5-2. 볼트별 진행

날짜	볼트	산출물
7/24	킵오프 + PvP 설계	기획서, 레포 구조
7/26	PvP 코어	실시간 매칭, 게이지
7/27	UX 전 씬 설계	명세 11종
7/28 ~ 7/30	전투 구현	스킬바, 쿨다운
8/3 ~ 8/4	싱글 코어 루프	하강, 강화, 방치
8/5 ~ 8/6	통합 + QA	카드, 타락도
8/7	로스터 · 리컬러	캐릭터 4종
8/8	BGM 분리, 삽화	음악 2곡, 인물
8/9	유저 신고 4건 수정 + 배포	라이브 확인

5-3. 이 방법론이 실제로 준 것

기존 방식이라면	AI-DLC로 실제로
설계 회의 → 문서 → 구현에 며칠	설계 문서는 AI가 초안, 사람은 결정 게이트만 봅니다
"왜 이렇게 했지?"를 되짚을 수 없습니다	결정 근거가 코드 주석과 문서에 남습니다
테스트를 나중에 몰아서	기능마다 같이 나와 1,657개가 쌓였습니다
QA 후 다음 스프린트에 수정	신고받은 날 고쳐서 배포합니다

6. 에셋 출처와 라이선스

상용 판매를 포함한 모든 용도로 사용 가능한 것만 썼습니다. 전체 기록은 레포의 `CREDITS.md`에 있고, **스크립트가 자동 생성합니다** — 손으로 적으면 에셋을 바꾼 날 문서만 옛것으로 남습니다.

트랙	라이선스	표기 의무	대상
CC0 팩	CC0	없음	효과음, 배경 요소 (Kenney)
chierit	CC-BY 4.0	있음	주인공 스프라이트
Kevin MacLeod	CC-BY 4.0	있음	BGM 2곡
한정호, 임연옥, 황덕현 생성	소유권 한정호, 임연옥, 황덕현, 은승현 (인물 이미지 생성)	해당 없음	인물 · 배경 삽화, 이펙트, 아이콘
Galmuri11	SIL OFL 1.1	표기	도트 한글 폰트

금지한 것은 코드로 막습니다

CC-BY-SA(라이선스 전파형) · 비상업용 · 재배포 금지는 쓰지 않습니다. `import_chars.py` 와 `fetch_bgm.sh` 가 라이선스 필드를 검사해서 막습니다 — 사람이 기억하는 규칙은 지켜지지 않습니다.

변경 고지 (CC-BY 의무): BGM 2곡은 모노 44.1kHz 변환, 루프 구간 절단, 이음새 등파워 크로스페이드, 56k 인코딩을 거쳤습니다. 스프라이트는 크롭 · 재배치 · 색 재배치를 했습니다.

7. 맺음 — 심사위원께

저희가 이 사전과제에서 보여드리고 싶은 것은 게임 하나가 아니라 **게임을 만드는 방식**입니다.

AI로 게임을 만드는 팀은 많아질 것입니다. 차이는 **AI를 몇 개 썼는지가 아니라, AI가 빠르게 만든 것을 어떻게 무너지지 않게 붙잡는 지**에서 납니다. AI-DLC는 그 질문에 대한 AWS의 답이고, 저희는 그것을 게임 개발에 적용해 12일 만에 배포까지 끌고 갔습니다.

판단해 주실 근거를 세 가지로 정리했습니다.

#	근거	확인 방법
1	지금 플레이해 보실 수 있습니다	abyss.agenticmind.cloud — 설치도 로그인도 없습니다
2	과정이 전부 남아 있습니다	설계 문서 16개, 커밋 131개, 테스트 1,657개. 결정 근거까지 코드에 있습니다
3	실패도 적었습니다	3D 리깅 폐기, 프롬프트 3회 실패, 테스트는 통과했는데 화면이 틀린 사건 3건

본 해커톤에 진출하면

이 방법론을 **팀이 커진 상태에서** 검증해 보고 싶습니다. 볼트 주기를 하루에서 반나절로 줄이고, 결정 게이트를 여러 사람이 나눠 쥐는 형태가 어디까지 성립하는지가 다음 질문입니다.

참고 자료

자료	링크
AWS, <i>AI-Driven Development Life Cycle</i>	aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/
AI-DLC 소개	prod.d13rzhkk8cj2z0.amplifyapp.com
<code>awslabs/aidlc-workflows</code>	github.com/awslabs/aidlc-workflows